SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Technical Presentation

Comparative Analysis of Polynomial and Logarithmic Algorithms

BY
k.jaya venkata sai
192525182

Supervisor
Dr.A.Sairam
Professor

Algorithm Overview

Algorithms Considered

Logarithmic Algorithm – O(log n)

- Example: Binary Search
- Divides the search space into halves.

Linear Algorithm – O(n)

- Example: Linear Search
- Checks each element sequentially.

Polynomial Algorithm – $O(n^2)$

- Example: Bubble Sort
- Compares every element with others repeatedly.

Problem Statement

- In today's applications, the amount of data is increasing rapidly, making algorithm efficiency an important factor. Different algorithms require different amounts of time to solve the same problem as the input size grows.
- **Logarithmic (O(log n))**, **Linear (O(n))**, and **Polynomial ($O(n^2)$)** algorithms show different performance characteristics.
- The challenge is to identify which algorithm is more efficient and scalable for different input sizes.
- For this analysis, we assume input sizes ranging from **100 to 100,000 elements.**
- The algorithms are compared based on **execution time, number of operations, memory usage, and scalability.** The goal is to determine the most suitable algorithm for real-world applications by using comparison tables and graphical analysis.

Algorithm / Pseudocode

Binary Search (O(log n))

```
low = 0
high = n-1

while(low <= high)
{
 mid = (low + high)/2

 if(key == a[mid])
     return mid;

 else if(key < a[mid])
     high = mid-1;

 else
     low = mid+1;
}
```

Linear Search (O(n))

```
for(i=0;i<n;i++)
{
   if(a[i]==key)
      return i;
}
```

Bubble Sort (O(n²))

```
for(i=0;i<n-1;i++)
{
   for(j=0;j<n-i-1;j++)
   {
      if(a[j]>a[j+1])
          swap();
   }
}
```

Working Example

Input

```
Array = {5,10,15,20,25,30,35}
Key = 20
```

Binary Search

```
Middle = 20

Found in 1 comparison
```

Linear Search

```
5
10
15
20

Found after 4 comparisons
```

Bubble Sort

Input	Output
5 3 2 4	2 3 4 5

Complexity Analysis

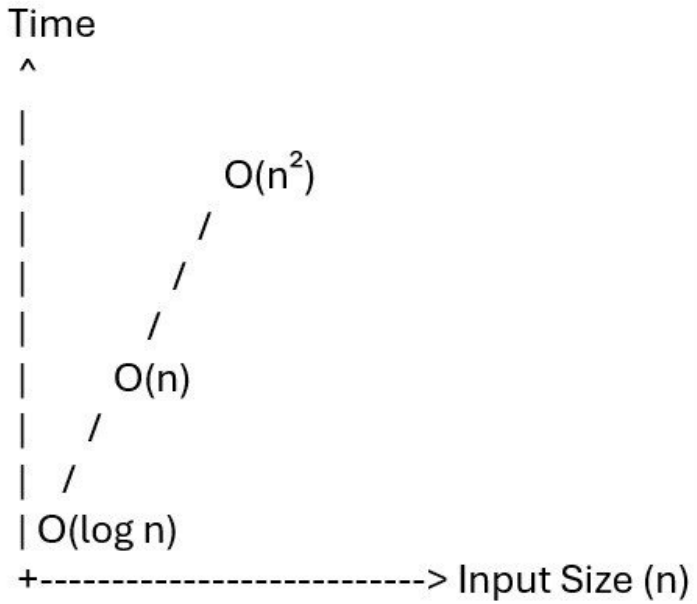
Algorithm	Best	Average	Worst	Space
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Linear Search	O(1)	O(n)	O(n)	O(1)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)

Comparative Analysis

Feature	O(log n)	O(n)	$O(n^2)$
Example	Binary Search	Linear Search	Bubble Sort
Scalability	Excellent	Good	Poor
Performance	Fastest	Moderate	Slow
Large Input	Best	Acceptable	Not Recommended
Time Growth	Very Slow	Linear	Rapid

Performance Graph

- The **X-axis** represents the **Input Size (n)**, and the **Y-axis** represents the **Execution Time** of the algorithms.
- The **O(log n)** curve grows very slowly, showing that logarithmic algorithms are the **most efficient** and perform well even for large datasets.
- The **O(n)** curve increases at a constant rate with the input size, providing **moderate performance** and good scalability. The **$O(n^2)$** curve rises steeply as the input size increases, indicating that polynomial algorithms become **less efficient** for large datasets.
- **Overall, the graph shows that O(log n) is the best choice for large inputs, O(n) offers acceptable performance, and $O(n^2)$ is suitable only for small datasets due to its higher execution time.**



Applications

Binary Search

- Database searching
- Dictionary search
- Phone contacts
- Search engines

Linear Search

- Small datasets
- Unsorted lists
- File searching

Bubble Sort

- Educational purposes
- Small datasets
- Algorithm learning

Applications

Binary Search

- Database searching
- Dictionary search
- Phone contacts
- Search engines

Linear Search

- Small datasets
- Unsorted lists
- File searching

Bubble Sort

- Educational purposes
- Small datasets
- Algorithm learning

Results & Discussion

Result

- **Binary Search (O(log n))** provides the best performance but requires sorted data.
- **Linear Search (O(n))** is simple and works well for unsorted or small datasets.
- **Bubble Sort ($O(n^2)$)** becomes inefficient as the input size grows.

Discussion

- For large datasets, logarithmic algorithms are highly scalable.
- Linear algorithms offer a balance between simplicity and performance.
- Polynomial algorithms should generally be avoided for large-scale problems

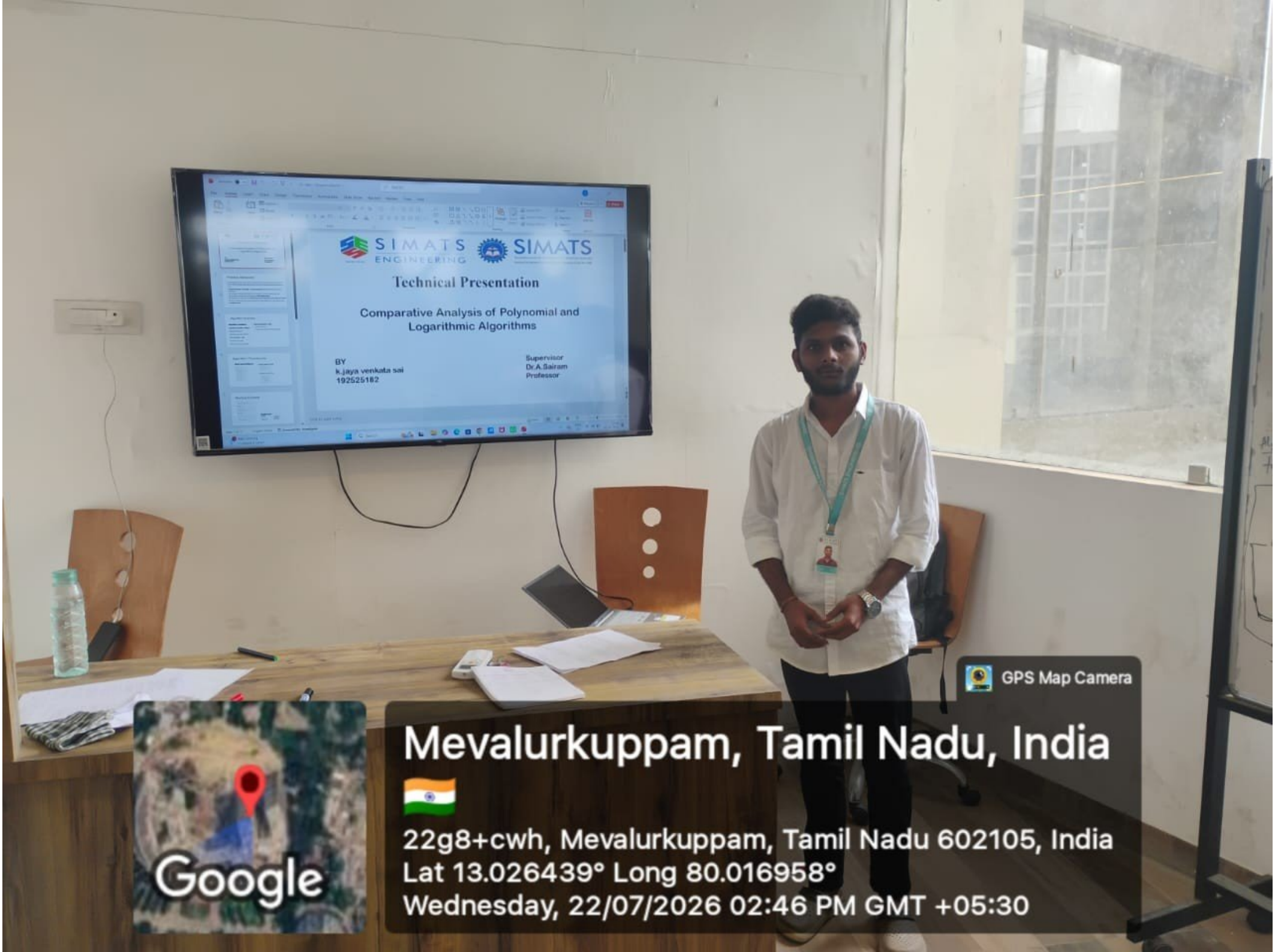
Conclusion & References

Conclusion

- Time complexity significantly impacts algorithm performance.
- **O(log n)** algorithms are the most scalable.
- **O(n)** algorithms are suitable for moderate-sized problems.
- **$O(n^2)$** algorithms are mainly useful for learning and small datasets.

References (IEEE)

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- S. S. Skiena, *The Algorithm Design Manual*, 3rd ed., Springer, 2020.
- M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 7th ed., Wiley, 2022.
- A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 4th ed., Pearson, 2024.


SIMATS ENGINEERING
SIMATS
Technical Presentation
Comparative Analysis of Polynomial and Logarithmic Algorithms
BY
k.jaya venkata sai
192525182
Supervisor
Dr.A.Sairam
Professor
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026439° Long 80.016958°
Wednesday, 22/07/2026 02:46 PM GMT +05:30
Google